

LAB 1 예비보고서

조합논리회로 설계와 사전 시뮬레이션 검증

Logic Gates · Adder · Subtractor · Comparator · MUX/DEMUX · Encoder/Decoder · 7-Segment

과목	이름	학번	작성일
전기전자컴퓨터설계실험 2	김태이	2023440158	2026-09-14

GitHub 저장소: https://github.com/tae2yi/UOS_ece2/tree/master/lab1

목차

1. 실험목적
2. 배경이론
3. 실험방법
4. 사전작업
5. 결론
6. 참고문헌

1. 실험목적

본 실험의 목적은 다음 세 가지다.

- 조합논리회로 10 종(기본 게이트, 가산기, 감산기, 비교기, 멀티플렉서, 디멀티플렉서, 인코더, 디코더, 7 세그먼트 디코더)의 동작 원리를 이해하고, 각 회로의 입출력 관계를 진리표 수준에서 예측할 수 있다.
- Verilog HDL 로 각 회로를 직접 기술하고, 기대값을 스스로 계산해 비교하는 자기검사(self-checking) 테스트벤치를 작성하여 설계의 기능적 정확성을 검증한다.
- FPGA 보드에 구현하기 전 단계로서 VS Code 와 Icarus Verilog 기반의 사전 시뮬레이션 환경을 구축하고, 파형(VCD)과 로그를 통해 결과를 해석하는 절차를 익힌다.

사전 시뮬레이션 단계에서 기능 오류를 미리 제거하면, 이후 Vivado 합성·구현과 실제 보드 기록 단계에서 발생하는 문제를 배선·제약 조건 문제로 좁혀서 진단할 수 있다. 본 보고서는 이 사전 검증 단계까지의 설계와 결과를 정리한 것이다.

대상 회로

Lab	회로	설계 top 모듈	입력 → 출력
01	AND/OR/XOR 로직 게이트	logic_gate	a, b (각 1비트) → x=AND, y=OR, z=XOR (각 1비트)
02	전가산기 (Full Adder)	full_adder	a, b, cin (각 1비트) → s(합), cout(자리올림) (각 1비트)
03	4비트 가산기	adder_4bit	a[3:0], b[3:0] → s[3:0](합), cout(자리올림)
04	4비트 감산기	sub_4bit	a[3:0], b[3:0] → d[3:0](차), bor(빌림)
05	4비트 비교기	compare_4	a[3:0], b[3:0] → o[2:0] (o[2]=a>b, o[1]=a==b, o[0]=a<b)
06	4:1 멀티플렉서	mux_4x1	i[3:0], s[1:0] → z (1비트)
07	1:8 디멀티플렉서	demux_1x8	i(1비트), s[2:0] → o[7:0]
08	8:3 인코더	encoder8x3	i[7:0] → a[2:0] (reg)
09	3:8 디코더	decoder3x8	a, b, c (각 1비트) → o[7:0]
10	7세그먼트 디코더	seg_decoder	bcd[3:0] → seg_data[7:0] ({a,b,c,d,e,f,g,dp} 순서, 1=점등, dp는 항상 0)

2. 배경이론

이번 실습의 10개 회로는 모두 조합논리회로(combinational logic circuit)다. 클럭이나 기억 소자 없이 현재 입력만으로 출력이 결정되며, 입력이 바뀌면 게이트 전파 지연(수 ns) 뒤 곧바로 출력이 갱신된다. 따라서 보드에서 스위치를 꺾으면 LED가 즉시 반응하고, 시뮬레이션에서도 입력을 바꾼 뒤 일정 시간만 기다리면 안정된 출력을 읽을 수 있다. 아래에서 10개 회로를 네 계열로 나누어 설명한다.

2.1 기본 논리 게이트 (AND · OR · XOR)

해당 실습: Lab 01

AND는 두 입력이 모두 1일 때만 1, OR는 하나 이상이 1이면 1, XOR는 두 입력이 서로 다를 때만 1이 된다. 세 연산을 같은 입력 a, b 에 동시에 적용하면 세 출력이 서로 다른 조건에서 커지므로, 한 회로에서 세 게이트의 차이를 한 번에 관찰할 수 있다.

XOR는 "다름 검출기"로도 읽을 수 있다. a 와 b 가 같으면 0, 다르면 1이므로 뒤에 나오는 가산기의 합(sum) 비트와 비교기의 "같음" 판정에서 다시 등장한다.

$a \ b$	$x = a \text{ AND } b$	$y = a \text{ OR } b$	$z = a \text{ XOR } b$
00	0	0	0
01	0	1	1
10	0	1	1
11	1	1	0

2.2 가산기 — 자리올림(carry)의 의미

해당 실습: Lab 02, Lab 03

반가산기(half adder)는 두 비트를 더해 합 $s = a \text{ XOR } b$ 와 자리올림 $c = a \text{ AND } b$ 를 만든다. 아래 자리에서 올라온 자리올림(cin)까지 받아야 하는 전가산기(full adder)는 반가산기 두 개를 직렬로 연결하고, 두 반가산기가 만든 자리올림을 OR로 합쳐 최종 cout을 만든다. 세 입력의 정수 합(0~3)이 2진수로 {cout, s}에 그대로 나타난다.

4비트 가산기는 이 개념을 4자리로 확장한 것이다. a 와 b 가 각각 0~15이므로 합은 최대 30이 되는데, 30은 4비트로 표현할 수 없다. 그래서 5번째 자리(16의 자리)를 cout으로 분리한다. 예를 들어 $15+1=16$ 이면 하위 4비트 s 는 0000이 되고 cout만 1이 된다. cout이 없으면 결과 0000을 보고 "0"인지 "16"인지 구분할 수 없으므로, cout은 4비트를 넘어선 정보를 보존하는 필수 출력이다.

2.3 감산기 — 빌림(borrow)의 의미와 2의 보수

해당 실습: Lab 04

감산기의 borrow는 "2의 보수로 해석하라는 표시" 이전에, 뺄셈 과정에서 실제로 발생하는 빌림이다. 10진수 뺄셈에서 아랫자리가 모자라면 윗자리에서 10을 빌려오듯, 2진수에서는 윗자리에서 2를 빌려온다. 중간 자리에서 생긴 빌림은 다음 자리가 갚아주므로 문제가 되지 않는다. 문제가 되는 것은 최상위 비트에서 빌림이 발생했는데 더 이상 빌려올 자리가 없는 경우이며, 이때 밖으로 새어 나가는 빌림이 곧 bor 출력이다.

아래 표는 0-2를 4비트로 계산하는 과정이다. 최상위 비트(bit3)에서 빌림이 밖으로 나가므로 bor=1이 되고, 결과 d=1110(=14)은 4비트 2의 보수로 읽으면 -2다.

비트	a	b	빌림 입력	계산	d	빌림 출력
0	0	0	0	0-0	0	0
1	0	1	0	2-1 (빌림 발생)	1	1
2	0	0	1	2-0-1 (빌림 발생)	1	1
3 (최상위)	0	0	1	2-0-1 (빌림 발생)	1	1 → bor

3-6도 같은 방식으로 bit2에서 빌림이 생기고 bit3에서 다시 빌림이 밖으로 나가 bor=1, d=1101(=13, 2의 보수로 -3)이 된다. 결국 최상위에서 빌림이 나가는 경우는 $a < b$ 일 때뿐이므로, 본 실습의 RTL은 비트별 빌림 연쇄를 직접 구현하는 대신 $\text{assign bor} = a < b$ 로 같은 결과를 얻는다.

정리하면 cout과 bor는 대칭 관계다. cout은 4비트 위로 넘친 자리올림이고, bor는 4비트 아래로 모자란 빌림이다. 둘 다 4비트 결과만으로는 알 수 없는 정보를 알려주는 다섯 번째 비트 역할을 한다.

2.4 비교기 — 삼분법과 one-hot 출력

해당 실습: Lab 05

비교기는 연산 결과를 만들지 않고 두 수의 관계만 판정한다. 두 수의 관계는 크다.같다.작다 셋 중 정확히 하나이므로(삼분법), 출력 3비트 중 항상 하나만 1이 되는 one-hot 형태로 인코딩하면 디코더 없이 LED 3개에 직접 연결할 수 있다.

비교 자체는 사람이 여러 자릿수를 비교하는 방식과 같다. 최상위 비트부터 내려가며 처음으로 값이 다른 자리에서 승부가 결정되고, 그 아래 자리는 볼 필요가 없다. 예를 들어 $a=1010$, $b=1001$ 이면 bit3과 bit2는 같고 bit1에서 a가 1, b가 0이므로 그 시점에 $a > b$ 가 확정되며 bit0의 값은 결과에 영향을 주지 않는다. 모든 비트가 끝까지

같으면 "같음"이 된다.

2.5 멀티플렉서와 디멀스 — 데이터와 주소의 구분

해당 실습: Lab 06, Lab 07

멀티플렉서(mux)와 디멀스(demux)는 계산을 하지 않고 신호의 경로를 정하는 라우팅 회로다. 앞선 가산기·비교기가 입력 비트 전체를 결과 계산에 사용한 것과 달리, 이 회로들은 선택된 하나의 선만 결과에 관여한다.

핵심은 입력을 두 종류로 나누어 이해하는 것이다. 데이터 입력(mux의 i)은 그 값 자체가 출력이 될 수 있는 후보이고, 선택 입력(s)은 값이 아니라 "몇 번째 후보를 볼 것인가"를 가리키는 주소다. 프로그래밍의 배열로 비유하면 i 는 배열, s 는 인덱스이며, 인덱스 값 자체는 결과에 직접 나타나지 않고 어느 원소를 꺼낼지만 결정한다.

선택 입력의 비트 폭은 후보 개수로 결정된다. 후보가 4개면 2비트($2^2=4$), 8개면 3비트($2^3=8$)가 필요하다. 그래서 4:1 mux의 s 는 2비트, 1:8 demux의 s 는 3비트다.

디멀스는 mux의 반대 방향이다. mux가 여러 입력 중 하나를 골라 하나의 출력으로 합류시킨다면, demux는 입력 하나를 s 가 가리키는 여러 출력 중 하나로 내보내고 나머지 출력은 0으로 둔다. 데이터 입력이 0이면 어느 출력도 켜지지 않는다. 실제 시스템에서는 두 회로를 짝지어 여러 장치가 하나의 신호선을 공유하도록 만드는 데 쓴다.

2.6 인코더와 디코더 — one-hot 과 이진 주소의 상호 변환

해당 실습: Lab 08, Lab 09

인코더는 8개의 입력선 중 활성화된 한 자리의 위치를 3비트 이진수로 압축하고, 디코더는 반대로 3비트 주소를 받아 8개 출력 중 해당 위치 하나만 켜준다. 즉 "어느 선이 켜졌는가"라는 위치 정보와 "몇 번인가"라는 숫자 정보를 서로 변환하는 한 쌍이다.

두 방향은 대칭처럼 보이지만 정보량 측면에서 비대칭이다. 디코더는 입력 8가지(000~111) 각각이 유효한 출력 하나에 정확히 대응하므로 애매한 경우가 없다. 반면 인코더의 입력은 8비트라 256가지 조합이 가능한데 그중 one-hot인 8가지만 유효하다. 나머지(0개 또는 2개 이상 켜진 경우)는 정의할 출력이 없어 default로 000을 내보낸다.

이 때문에 본 실습의 인코더에는 구조적 모호성이 있다. 아무것도 켜지지 않은 경우, 두 개 이상 켜진 무효 입력, 그리고 정상적으로 0번($i[7]$)이 켜진 경우가 모두 출력 000으로 같게 나타나 출력만으로는 구분할 수 없다. 또한 여러 입력이 동시에 켜졌을 때 우선순위를 정해 하나를 선택하지 않으므로 우선순위 인코더(priority encoder)가 아니다. 실제 상용 인코더 IC는 이 문제를 해결하기 위해 출력이 유효한지 알려주는 별도의 신호를 추가로 둔다.

상황	$i[7:0]$	출력 $a[2:0]$
$i[7]$ 만 켜짐 (정상)	10000000	000

상황	i[7:0]	출력 a[2:0]
아무것도 켜지지 않음	00000000	000
두 개 동시 (무효)	00000011	000

2.7 7 세그먼트 디코더 — 획 조합으로 문자 표시

해당 실습: Lab 10

7세그먼트 디코더도 디코더이지만 출력 형태가 다르다. 3:8 디코더는 항상 출력 하나만 켜지는 one-hot이었던 반면, 7세그먼트 디코더는 a~g 일곱 획 중 여러 개를 동시에 켜서 사람이 읽을 수 있는 문자 모양을 만든다. 예를 들어 0은 가운데 획 g를 뺀 여섯 획, 1은 오른쪽 세로 두 획(b, c), 8은 일곱 획 전부를 켜다. 출력 비트가 "몇 번째 선인가"가 아니라 "어느 획인가"를 뜻하므로 패턴이 불규칙하다.

입력이 4비트이므로 0~15의 16가지 값을 모두 표시할 수 있고, 10~15는 A, b, C, d, E, F로 정의한다. 즉 이름은 BCD이지만 실제로는 16진수 한 자리를 표시하는 디코더다. 10~15에 소문자 b와 d를 쓰는 것은 관행인데, 7세그먼트로 대문자 B를 그리면 8과, 대문자 D를 그리면 0과 모양이 같아져 구분할 수 없기 때문이다. 소수점(dp) 획은 본 실습에서 사용하지 않으므로 항상 0으로 둔다.

3. 실험방법

3.1 개발 환경

사전 시뮬레이션은 Vivado 없이 수행할 수 있다. 본 실험에서는 macOS 호스트에서 VS Code와 Icarus Verilog를 사용했고, Vivado는 Docker 컨테이너에 별도로 설치하여 2부(구현 단계)에서 사용한다. 호스트 디렉터리를 컨테이너에 바인드 마운트했으므로 동일한 소스를 양쪽에서 그대로 사용할 수 있다.

도구	버전	용도
Git	2.55.0	템플릿 clone 및 소스 커밋 이력 관리
Python	3.9.6	작업 실행 스크립트(tools/lab1.py) 구동
Icarus Verilog	13.0 (stable)	Verilog 컴파일(iverilog) 및 시뮬레이션 실행(vvp)
VS Code + VaporView	-	코드 편집, 작업 실행, VCD 파형 확인

3.2 프로젝트 구조

각 회로마다 동일한 템플릿 저장소를 실습 이름으로 clone하여 독립된 프로젝트로 구성했다. 프로젝트의 주요 구성은 다음과 같다.

경로	내용
src/	설계 RTL(.v). 하위 모듈을 추가하면 simulation.json의 sources에도 등록한다.
sim/	자기검사 테스트벤치(.sv). 설계 top과 구분하기 위해 tb_ 접두사를 사용한다.
constraints/	보드 핀 배정을 기술한 XDC. 기능 시뮬레이션에서는 사용하지 않는다.
simulation.json	sources · testbench · simulation_top을 지정하는 실행 설정 파일.
build/sim/	실행별 compile.log · simulation.log · wave.vcd가 생성되는 출력 폴더.
reports/, evidence/	제출용 레포트와 로그·파형·캡처 증빙 자료.

본 보고서에서 다루는 10개 실습의 전체 소스와 증빙 자료는 다음 저장소에 공개되어 있으며, 4장의 각 실습마다 해당 파일로 연결되는 링크를 함께 표기했다.

https://github.com/tae2yi/UOS_ece2/tree/master/lab1

3.3 작업 실행 절차

VS Code의 Terminal → Run Task 메뉴에서 다음 세 작업을 순서대로 실행한다.

- 01 Check tools — Python · Git · Icarus Verilog 의 설치 여부와 버전을 확인한다.
- 02 Simulate — simulation.json 에 지정한 RTL 과 테스트벤치를 컴파일(iverilog)하고 실행(vvp)한다.
- 03 Open waveform — 이번 실행에서 생성된 build/sim/wave.vcd 를 VaporView 로 연다.

3.4 자기검사 테스트벤치와 PASS 판정 기준

본 실험의 합격 판정은 도구의 실행 성공 여부가 아니라 테스트벤치가 스스로 내리는 검사 결과를 근거로 한다. 이 구분은 중요하므로 명확히 정리한다.

03 Open waveform이 표시하는 SIMULATED 상태는 컴파일·실행·파형 생성이 끝났다는 의미일 뿐이며, 회로가 정답이라는 판정이 아니다. 실제로 실행 결과 파일 build/sim/result.json에도 functional_correctness 항목이 NOT_ASSESSED로 기록된다. 도구는 기능의 옳고 그름을 판단하지 않는다.

대신 각 테스트벤치는 다음 구조로 스스로 정답을 검사하도록 작성했다.

- 반복문으로 입력 조합을 순서대로 인가하고, 각 조합마다 기대값을 테스트벤치가 독립적으로 계산한다.
- 일정 시간(10ns) 뒤 실제 출력과 기대값을 `!=` 연산자로 비교한다. `!=`는 0 과 1 뿐 아니라 X-Z 상태의 불일치도 검출한다.
- 불일치가 발생하면 `$fatal` 로 즉시 중단하고 입력 벡터·기대값·실제값을 로그에 출력한다. 따라서 중간에 하나라도 틀리면 실행 자체가 실패로 끝난다.
- 모든 조합을 통과하면 검사 횟수(`checked`)가 전체 케이스 수와 일치하는지 확인한 뒤 LAB1_PASS 문구를 출력하고 `$finish` 로 종료한다.
- 무한 루프 방지를 위해 100,000ns 시점에 watchdog 을 두어 그때까지 종료되지 않으면 실패 처리한다.

즉 로그에 나타나는 "LAB1_PASS <모듈명> cases=N"은 N개 입력 조합 전부가 자기검사를 통과했다는 뜻이며, 본 보고서의 모든 합격 판정은 이 문구를 근거로 한다.

3.5 파형 확인 방법

VaporView의 Netlist View에서 테스트벤치 계층을 펼쳐 입력·출력 신호를 추가하고, Zoom to Fit으로 전체 구간

을 맞춘 뒤 커서로 값을 읽는다. 값을 읽을 때는 신호가 전환되는 경계가 아니라 구간의 중간 지점에 커서를 두어야 안정된 값을 얻을 수 있다. 다중 비트 신호는 진법(bin/hex)을 지정해 표시한다.

3.6 코드 수정 실험 절차

테스트벤치가 실제로 오류를 검출할 수 있는지 확인하기 위해, 각 실습마다 교안이 지정한 지점을 의도적으로 한 줄 수정하여 오류를 주입하고 다음 절차로 검증했다.

- 정상 PASS 로그를 먼저 보관한다.
- 설계 RTL 의 지정된 부분을 한 줄만 바꾼다. 테스트벤치의 기대값은 수정하지 않는다.
- 02 Simulate 를 재실행하여 실패 로그에서 입력 벡터·기대값·실제값을 확인한다.
- 수정한 부분을 원래대로 복구하고 다시 실행하여 전체 PASS 와 새 파형을 확인한다.

이 절차는 "검사가 통과했다"는 사실만으로는 검사 자체가 유효한지 알 수 없다는 점을 보완한다. 오류를 주입했을 때 실제로 실패해야만 그 테스트벤치가 의미 있는 검증 도구임이 확인된다.

3.7 XDC 작성

각 실습의 constraints 폴더에 보드 핀 배정을 기술한 XDC를 작성했다. set_property PACKAGE_PIN은 설계의 논리적 포트를 FPGA 칩의 물리적 핀에 연결하고, set_property IOSTANDARD LVCMOS33은 해당 핀의 전기적 신호 규격을 3.3V CMOS로 지정한다. 버스의 특정 비트를 지정할 때는 get_ports {a[2]}와 같이 중괄호로 감싼다.

XDC는 Icarus 기능 시뮬레이션에서 읽지 않는다. 따라서 시뮬레이션이 통과했다는 사실이 핀 배치가 옳다는 근거는 되지 않으며, 핀 배정의 검증은 Vivado 구현 단계와 실제 보드에서 별도로 이루어져야 한다.

4. 사전작업

각 실습에 대해 설계 RTL·테스트벤치·XDC를 직접 작성하고, 실행 전 예상값을 먼저 정리한 뒤 시뮬레이션을 수행했다. 이어서 코드 수정 실험으로 테스트벤치의 검출 능력을 확인하고 원상 복구했다. 아래는 10개 실습의 작업 내용과 결과다.

4.1 01 · AND/OR/XOR 로직 게이트

설계 내용

포트: 입력 a, b (각 1비트) → 출력 x=AND, y=OR, z=XOR (각 1비트)

동작: assign 세 줄로 $a \& b$, $a | b$, $a \wedge b$ 를 동시에 계산하는 순수 조합회로이다. 세 출력 모두 입력이 바뀌면 지연 없이 즉시 갱신된다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_01_logic_gates

src/logic_gate.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_01_logic_gates/src/logic_gate.v

```
module logic_gate(input wire a,b, output wire x,y,z);
    assign x = a & b;
    assign y = a | b;
    assign z = a ^ b;
endmodule
```

실행 전 예상값

a b	x AND	y OR	z XOR
00	0	0	0
01	0	1	1
10	0	1	1
11	1	1	0

OR는 하나 이상 1, XOR는 두 입력이 서로 다를 때 1이다.

검사 조건과 테스트벤치

입력 조합: {a,b}=n (n=0..3)으로 00→01→10→11 순서로 대입, 각 10ns 유지

기대값 계산·비교: {x,y,z}를 기대값과 !=(4-state 비교)로 대조한다. 불일치 시 \$fatal로 즉시 중단하고 입력·기대값·실제값을 출력한다. checked 카운터가 4와 같은지 확인한 뒤 LAB1_PASS 문구를 출력하고 \$finish로 종료한다.

총 4개 조합 × 10ns = 40ns에 정상 종료.

sim/tb_logic_gate_modern.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_01_logic_gates/sim/tb_logic_gate_modern.sv

```
`timescale 1ns/1ps
module tb_logic_gate_modern;
  reg a,b; wire x,y,z; logic_gate dut(a,b,x,y,z);
  integer n,checked=0;
  reg [2:0] expected;
  initial begin
    $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);
    for(n=0;n<4;n=n+1) begin
      {a,b}=n;
      expected={(a & b),(a | b),(a ^ b)};
      #10;
      if ({x,y,z} != expected) begin
        $fatal(1,"FAIL logic_gate vector=%0d expected=%h actual=%h",n,expected,{x,y,z});
      end
      checked=checked+1;
    end
    if(checked!=4) $fatal(1,"Incomplete test");
    $display("LAB1_PASS logic_gate cases=%0d",checked);
    $finish;
  end
  initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS logic_gate cases=4

\$finish called at 40000 (1ps) → 40ns 종료

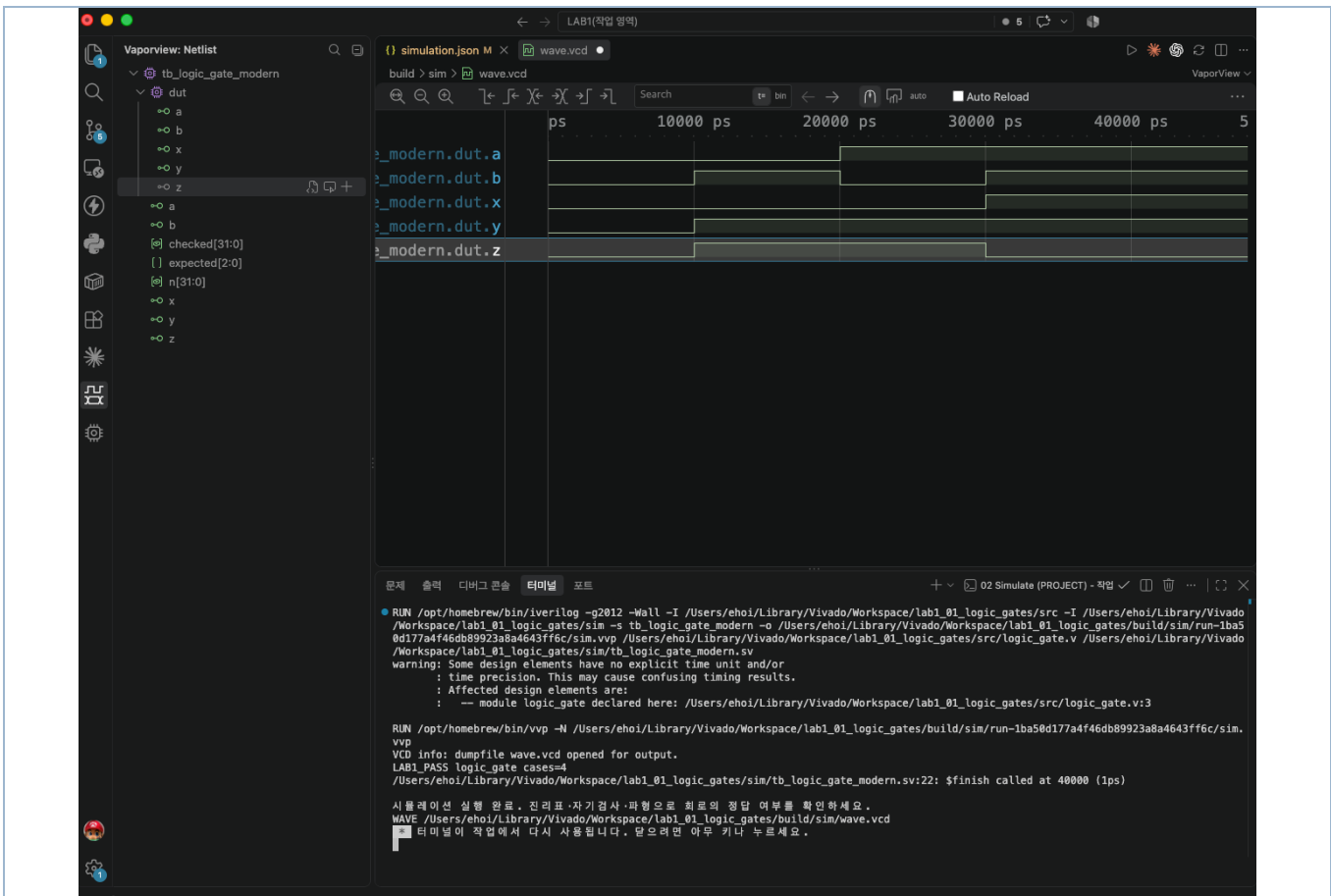


그림 4-1. logic_gate 파형 및 실행 로그. 터미널에서 "LAB1_PASS logic_gate cases=4" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a	b	x	y	z
5	0	0	0	0	0
15	0	1	0	1	1
25	1	0	0	1	1
35	1	1	1	1	0

캡처(사진)는 전체 40,000ps(=\$finish 시각)가 모두 보이는 Zoom to Fit 화면이므로, 위 4개 시간 전부 화면에서 직접 확인 가능하다.

코드 수정 실험

PDF 24~25쪽의 "코드 수정 실험"을 실제로 수행했다: logic_gate.v의 z를 XOR(^)에서 OR(|)로 바꾼 뒤(문법은 정상이지만 논리가 틀린 회로) 저장하고 02 Simulate를 재실행했다.

변경: `assign z = a ^ b;` → `assign z = a | b;`

실패 로그: FAIL logic_gate vector=3 expected=6 actual=7 (Time: 40000) — a=1,b=1에서 정상 z=0(기대값 {x,y,z}=110=6)이어야 하는데, OR로 바뀌며 z=1(실제값 111=7)이 되어 실패.

복구 확인: OR를 XOR로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS logic_gate cases=4, \$finish called at 40000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
b	N8	x	L4
y	M4	z	M2
a	K4		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/logic_gate.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_01_logic_gates/constraints/logic_gate.xdc

보드 확인 예정 입출력: a=K4, b=N8, x=L4, y=M4, z=M2, 다섯 포트 모두 IOSTANDARD LVCMOS33 (constraints/logic_gate.xdc에 작성 완료).

4.2 02 · 전가산기 (Full Adder)

설계 내용

포트: 입력 a, b, cin (각 1비트) → 출력 s(합), cout(자리올림) (각 1비트)

동작: half_adder(assign s=a^b, c=a&b) 두 개를 직렬로 연결한다. 첫 half_adder가 a와 b를 더하고, 두 번째 half_adder가 그 합과 cin을 다시 더한다. 두 half_adder에서 나온 carry 두 개를 OR로 합쳐 최종 cout을 만든다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_02_full_adder

src/half_adder.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_02_full_adder/src/half_adder.v

```
module half_adder(input wire a,b, output wire s,c);
    assign s = a ^ b;
    assign c = a & b;
endmodule
```

src/full_adder.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_02_full_adder/src/full_adder.v

```
module full_adder(input wire a,b,cin, output wire s,cout);
    wire partial_sum, carry_ab, carry_cin;
    half_adder first(.a(a),.b(b),.s(partial_sum),.c(carry_ab));
    half_adder second(.a(partial_sum),.b(cin),.s(s),.c(carry_cin));
    assign cout = carry_ab | carry_cin;
endmodule
```

실행 전 예상값

입력(a,b,cin)	예상 결과
000	cout=0, s=0
011	cout=1, s=0
111	cout=1, s=1

세 입력의 정수 합(0~3)이 2진수로 {cout,s}에 그대로 나타난다.

검사 조건과 테스트벤치

입력 조합: {a,b,cin}=n (n=0..7)으로 8가지 조합을 대입, 각 10ns 유지

기대값 계산·비교: 정수 덧셈 $\text{int}(a) + \text{int}(b) + \text{int}(cin)$ 을 기대값으로 계산해 {cout,s} 2비트와 비교(!==)한다. 불일치 시 \$fatal. checked=8을 확인한 뒤 LAB1_PASS 출력, \$finish.

총 8개 조합 × 10ns = 80ns에 정상 종료.

sim/tb_full_adder.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_02_full_adder/sim/tb_full_adder.sv

```
`timescale 1ns/1ps
module tb_full_adder;
    reg a,b,cin; wire s,cout; full_adder dut(a,b,cin,s,cout);
    integer n,checked=0;
    reg [1:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_full_adder);
        for(n=0;n<8;n=n+1) begin
            {a,b,cin}=n;
            expected=int'(a)+int'(b)+int'(cin);
        end
    end
endmodule
```

```

#10;
if ({cout,s} != expected) begin
    $fatal(1,"FAIL full_adder vector=%0d expected=%h actual=%h",n,expected,{cout,s});
end
checked=checked+1;
end
if(checked!=8) $fatal(1,"Incomplete test");
$display("LAB1_PASS full_adder cases=%0d",checked);
$finish;
end
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

시뮬레이션 결과

LAB1_PASS full_adder cases=8

\$finish called at 80000 (1ps) → 80ns 종료

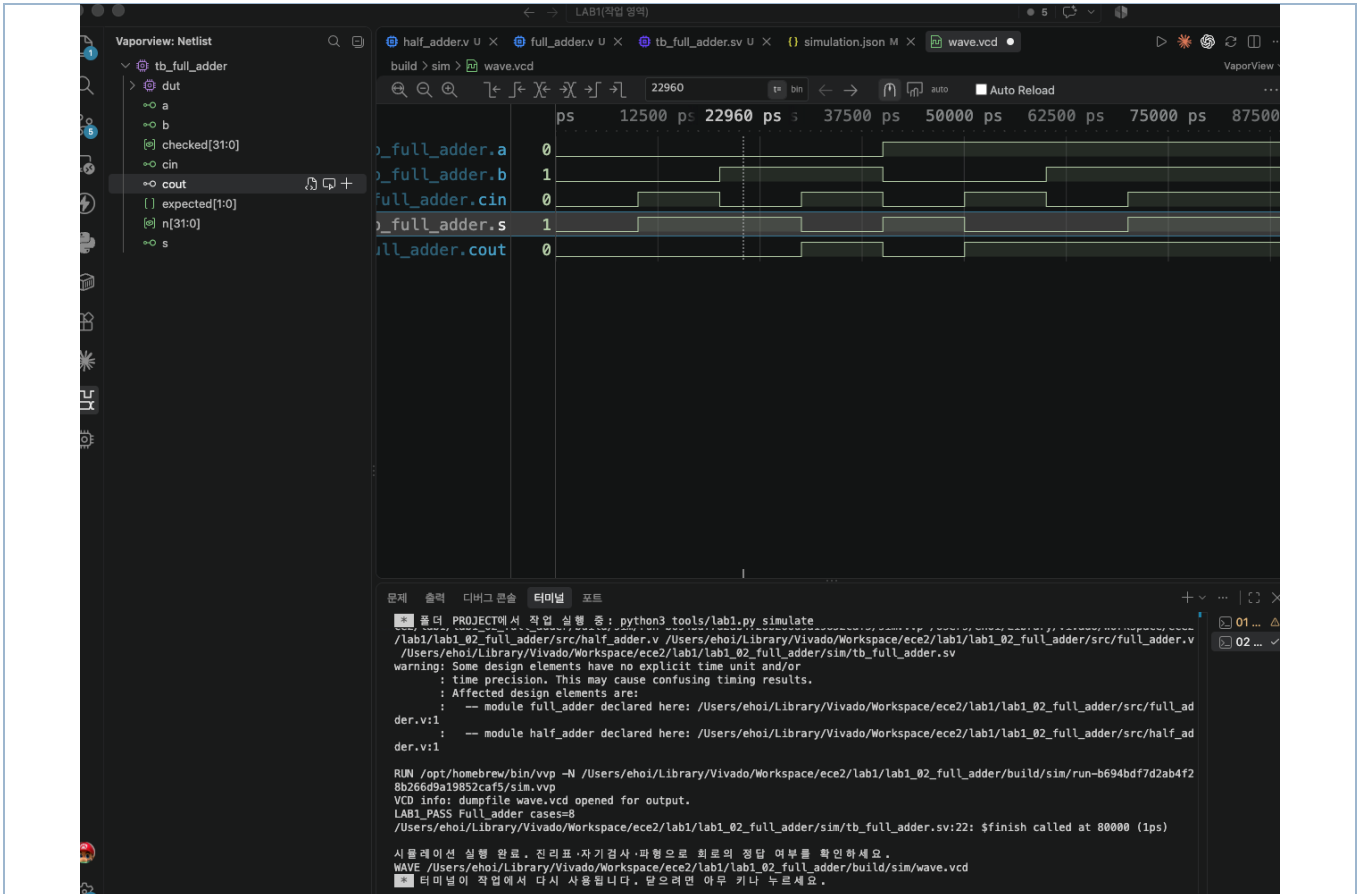


그림 4-2. full_adder 파형 및 실행 로그. 터미널에서 "LAB1_PASS full_adder cases=8" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a	b	cin	s	cout
5	0	0	0	0	0
15	0	0	1	1	0
35	0	1	1	0	1
45	1	0	0	1	0
75	1	1	1	1	1

캡처(사진)는 전체 80,000ps 범위가 모두 보이며(커서가 22,960ps에 위치), 위 5개 시간 전부 화면에서 직접 확인 가능하다. ※

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: full_adder.v의 cout OR 연산자를 AND로 바꾼 뒤(assign cout = carry_ab & carry_cin;) 저장하고 02 Simulate를 재실행했다.

변경: assign cout = carry_ab | carry_cin; → assign cout = carry_ab & carry_cin;

실패 로그: FAIL full_adder vector=3 expected=2 actual=0 (Time: 40000) — a=0,b=1,cin=1에서 정상 carry=1(기대값 2=10)이어야 하는데 AND로 바뀌며 carry=0(실제값 0=00)이 되어 실패.

복구 확인: OR로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS full_adder cases=8, \$finish called at 80000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
a	K4	b	N8
cin	N4	cout	L4
s	M4		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/full_adder.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_02_full_adder/constraints/full_adder.xdc

보드 확인 예정 입출력: KEY1=a, KEY2=b, KEY3=cin. LED1=cout, LED2=s.

4.3 03 · 4 비트 가산기

설계 내용

포트: 입력 a[3:0], b[3:0] → 출력 s[3:0](합), cout(자리올림)

동작: assign {cout,s} = {1'b0,a} + {1'b0,b}; 두 4비트 수를 5비트로 확장해 더한 뒤, 상위 1비트를 cout으로 분리한다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_03_adder4

src/adder_4bit.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_03_adder4/src/adder_4bit.v

```
module adder_4bit(input wire [3:0] a,b, output wire [3:0] s, output wire cout);
    assign {cout,s} = {1'b0,a} + {1'b0,b};
endmodule
```

실행 전 예상값

입력	예상 결과
0+0	cout=0, s=0
1+15	cout=1, s=0
15+15	cout=1, s=14

1+15=16이므로 하위 4비트 합은 0000이 되어도 carry(cout)가 1이 되어 전체 값은 16임에 유의한다.

검사 조건과 테스트벤치

입력 조합: {a,b}=n (n=0..255)로 0~15 두 수의 모든 조합(16×16=256가지)을 대입

기대값 계산·비교: 기대값=(n/16)+(n%16) 정수합을 {cout,s} 5비트와 비교(!==). 불일치 시 \$fatal. checked=256

확인 후 LAB1_PASS, \$finish.

총 256개 조합 × 10ns = 2560ns에 정상 종료.

sim/tb_adder_4bit.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_03_adder4/sim/tb_adder_4bit.sv

```
`timescale 1ns/1ps
module tb_adder_4bit;
    reg [3:0] a,b; wire [3:0] s; wire cout; adder_4bit dut(a,b,s,cout);
    integer n,checked=0;
    reg [4:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_adder_4bit);
        for(n=0;n<256;n=n+1) begin
            {a,b}=n;
            expected=(n/16)+(n%16);
            #10;
            if ({cout,s} != expected) begin
                $fatal(1,"FAIL adder_4bit vector=%0d expected=%h actual=%h",n,expected,{cout,s});
            end
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS adder_4bit cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
```

endmodule

시뮬레이션 결과

LAB1_PASS adder_4bit cases=256

\$finish called at 2560000 (1ps) → 2560ns 종료

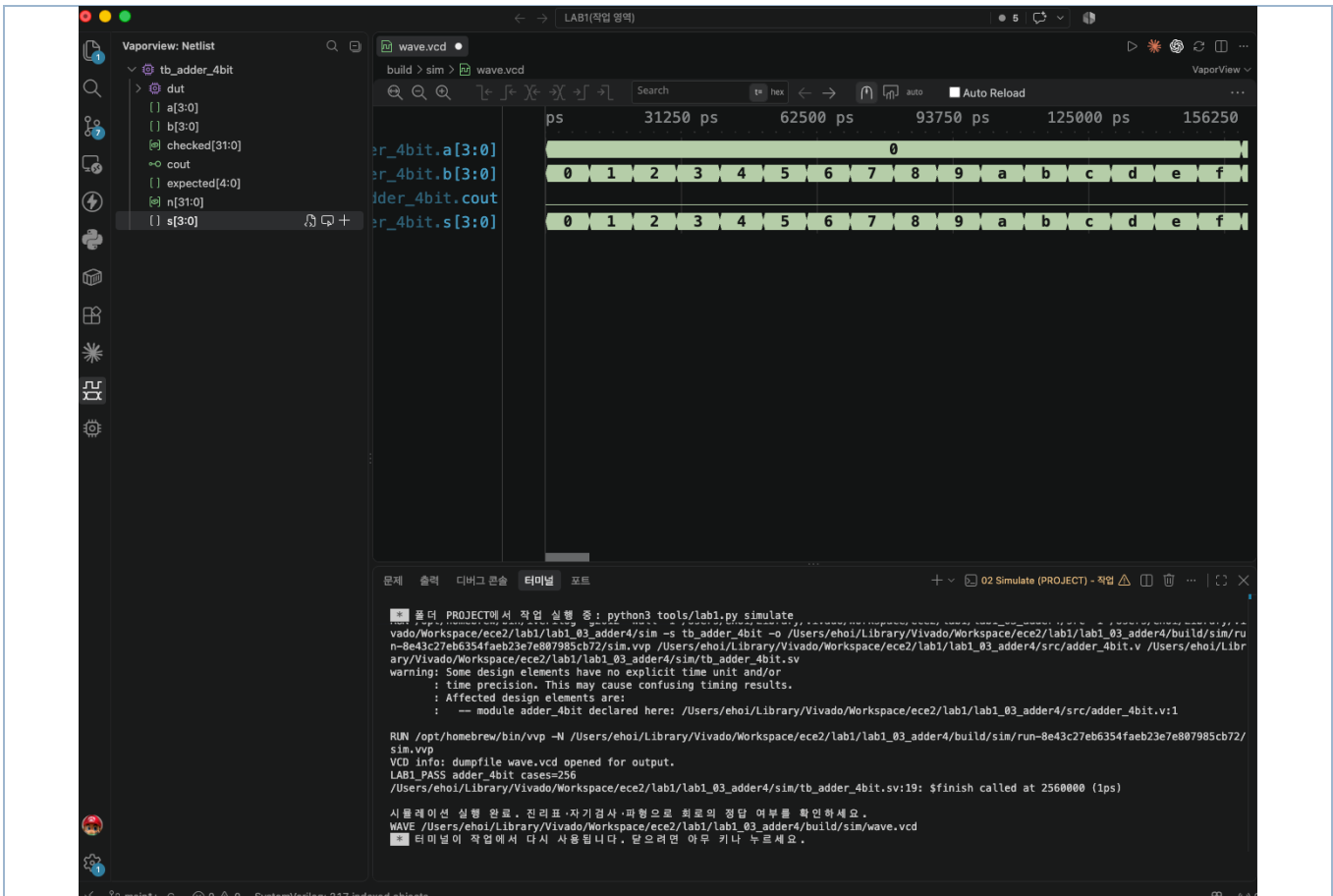


그림 4-3. adder_4bit 파형 및 실행 로그. 터미널에서 "LAB1_PASS adder_4bit cases=256" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a[3:0]	b[3:0]	s[3:0]	cout
5	0000	0000	0000	0
35	0000	0011	0011	0
65	0000	0110	0110	0
95	0000	1001	1001	0
155	0000	1111	1111	0

이 값은 캡처(사진)에 실제로 보이는 0~156,250ps 구간(전체 2,560,000ps 중 맨 앞부분, a=0000으로 고정된 첫 16케이스) 안에서 직접 읽은 것이다. 나머지 240케이스(a=0001~1111)는 화면에 없지만, checked==256 자기검사를 통과한 LAB1_PASS 로그로 전체 통과가 보증된다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: adder_4bit.v의 덧셈 연산자를 뺄셈으로 바꾼 뒤 저장하고 02

Simulate를 재실행했다.

변경: `assign {cout,s} = {1'b0,a} + {1'b0,b};` → `assign {cout,s} = {1'b0,a} - {1'b0,b};`

실패 로그: FAIL adder_4bit vector=1 expected=01 actual=1f (Time: 20000) — a=0,b=1에서 정상 결과는 cout=0,s=0001(=01)인데, 뺄셈으로 바뀌며 0-1의 4비트 wraparound로 cout=1,s=1111(=1f)이 되어 실패.

복구 확인: 덧셈으로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS adder_4bit cases=256, \$finish called at 2560000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
cout	L4	a[3]	Y1
a[2]	W3	a[1]	U2
a[0]	T1	b[3]	W4
b[2]	W1	b[1]	V4
b[0]	U4	s[3]	M4
s[1]	N7	s[2]	M2
s[0]	M7		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/adder_4bit.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_03_adder4/constraints/adder_4bit.xdc

보드 확인 예정 입출력: DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=cout, LED2-5=s[3:0].

4.4 04 · 4 비트 감산기

설계 내용

포트: 입력 a[3:0], b[3:0] → 출력 d[3:0](차), bor(빌림)

동작: assign d = a - b; assign bor = a < b; 두 4비트 수의 차를 그대로 대입하고, a<b일 때만 빌림(borrow)이 발생한다고 표시한다. a==b는 빌림이 없다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_04_subtractor4

src/sub_4bit.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_04_subtractor4/src/sub_4bit.v

```
module sub_4bit(input wire [3:0] a,b, output wire [3:0] d, output wire bor);
    assign d = a - b;
    assign bor = a < b; // Equal operands do NOT borrow.
endmodule
```

실행 전 예상값

입력	예상 결과
0-0	bor=0, d=0
0-1	bor=1, d=15
15-0	bor=0, d=15

0-1의 하위 4비트가 1111로 나타나며, 이를 4비트 2의 보수로 해석하면 -1이다. 동시에 bor=1로 a<b임을 나타낸다.

검사 조건과 테스트벤치

입력 조합: {a,b}=n (n=0..255)로 모든 조합을 대입

기대값 계산·비교: 기대값 = ((n/16)<(n%16) ? 16:0) | (((n/16)-(n%16))&15) 로 빌림 비트와 4비트 차를 함께 계산해 {bor,d}와 비교(!==). checked=256 확인 후 LAB1_PASS, \$finish.

총 256개 조합 × 10ns = 2560ns에 정상 종료.

sim/tb_sub_4bit.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_04_subtractor4/sim/tb_sub_4bit.sv

```
`timescale 1ns/1ps
module tb_sub_4bit;
    reg [3:0] a,b; wire [3:0] d; wire bor; sub_4bit dut(a,b,d,bor);
    integer n,checked=0;
    reg [4:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_sub_4bit);
        for(n=0;n<256;n=n+1) begin
            {a,b}=n;
            expected=(((n/16)<(n%16)) ? 16 : 0) | (((n/16)-(n%16)) & 15);
            #10;
            if ({bor,d} != expected) begin
                $fatal(1,"FAIL sub_4bit vector=%0d expected=%h actual=%h",n,expected,{bor,d});
            end
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS sub_4bit cases=%0d",checked);
        $finish;
    end
end
```

```
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS sub_4bit cases=256

\$finish called at 2560000 (1ps) → 2560ns 종료

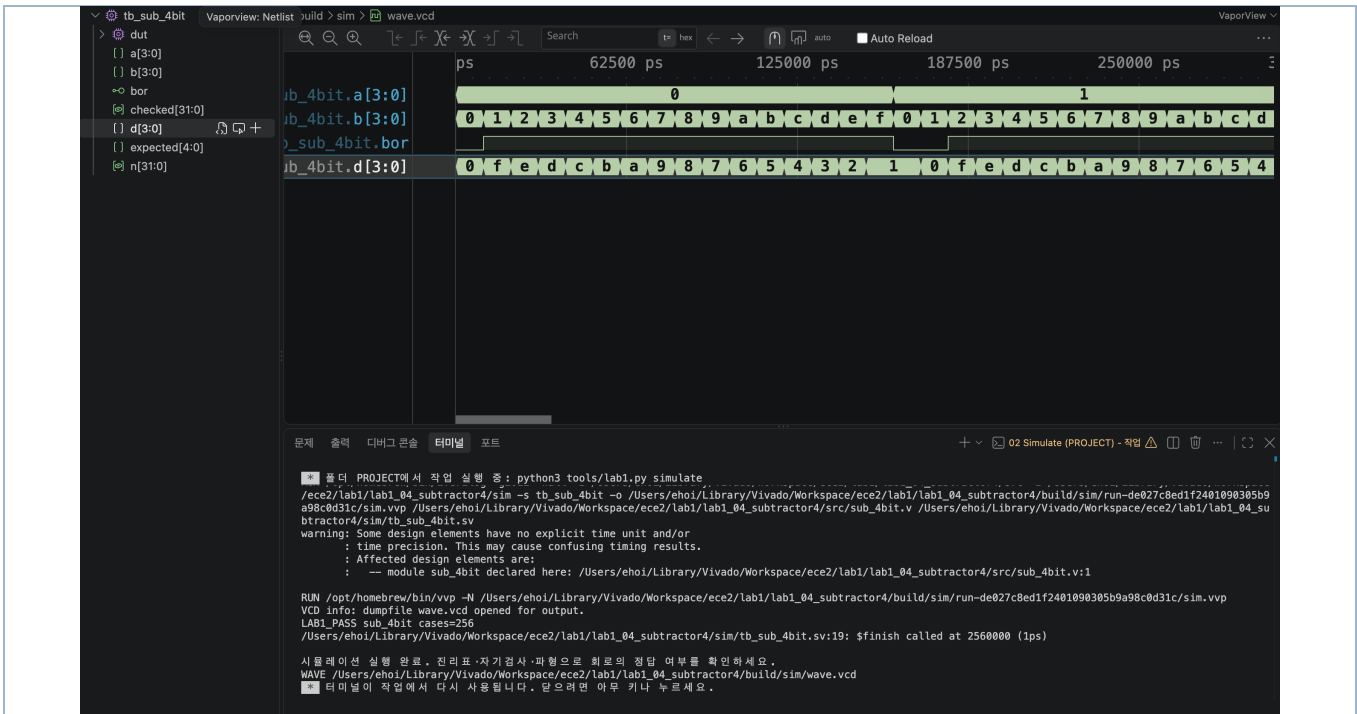


그림 4-4. sub_4bit 파형 및 실행 로그. 터미널에서 "LAB1_PASS sub_4bit cases=256" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a[3:0]	b[3:0]	d[3:0]	bor
5	0000	0000	0000	0
65	0000	0110	1010	1
125	0000	1100	0100	1
165	0001	0000	0001	0
245	0001	1000	1001	1

캡처(사진)에는 0~약 300,000ps 구간(전체 2,560,000ps 중 맨 앞부분)이 보이며, a가 0000→0001로 바뀌는 경계(160,000ps 부근)까지 포함되어 있어 위 표에 반영했다. 그 이후 케이스의 통과 여부는 화면이 아니라 checked==256 자기검사를 통과한 LAB1_PASS 로그로 확인했다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: sub_4bit.v의 borrow 비교에 등호를 추가해 $a \leq b$ 로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: assign bor = a < b; → assign bor = a <= b;

실패 로그: FAIL sub_4bit vector=0 expected=00 actual=10 (Time: 10000) — a=0,b=0에서 정상 borrow는 0(같

은 값은 빌림 없음)인데, $a \leq b$ 로 바뀌며 $bor=1$ (실제값 10)이 되어 실패.

복구 확인: $a < b$ 로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS sub_4bit cases=256, \$finish called at 2560000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
bor	L4	a[3]	Y1
a[2]	W3	a[1]	U2
a[0]	T1	b[3]	W4
b[2]	W1	b[0]	U4
d[3]	M4	d[1]	N7
b[1]	V4	d[2]	M2
d[0]	M7		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/sub_4bit.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_04_subtractor4/constraints/sub_4bit.xdc

보드 확인 예정 입출력: DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=bor, LED2-5=d[3:0].

4.5 05 · 4 비트 비교기

설계 내용

포트: 입력 a[3:0], b[3:0] → 출력 o[2:0] (o[2]=a>b, o[1]=a==b, o[0]=a<b)

동작: assign o = {a > b, a == b, a < b}; 세 비교 결과 중 정확히 하나만 1이 되도록 3비트로 인코딩한다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_05_comparator4

src/compare_4.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_05_comparator4/src/compare_4.v

```
module compare_4(input wire [3:0] a,b, output wire [2:0] o);
    assign o = {a > b, a == b, a < b};
endmodule
```

실행 전 예상값

입력	예상 결과
0 대 0	010
0 대 1	001
1 대 0	100

o=100은 a>b(큼), 010은 a==b(같음), 001은 a<b(작음)을 뜻한다.

검사 조건과 테스트벤치

입력 조합: {a,b}=n (n=0..255)로 모든 조합을 대입

기대값 계산·비교: 기대값 = (n/16)>(n%16) ? 4 : ((n/16)==(n%16) ? 2 : 1) 로 100/010/001을 계산해 o와 비교 (!==). checked=256 확인 후 LAB1_PASS, \$finish.

총 256개 조합 × 10ns = 2560ns에 정상 종료.

sim/tb_compare_4.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_05_comparator4/sim/tb_compare_4.sv

```
`timescale 1ns/1ps
module tb_compare_4;
    reg [3:0] a,b; wire [2:0] o; compare_4 dut(a,b,o);
    integer n,checked=0;
    reg [2:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_compare_4);
        for(n=0;n<256;n=n+1) begin
            {a,b}=n;
            expected=((n/16)>(n%16)) ? 4 : (((n/16)==(n%16)) ? 2 : 1);
            #10;
            if (o !== expected) begin
                $fatal(1,"FAIL compare_4 vector=%0d expected=%h actual=%h",n,expected,o);
            end
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS compare_4 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS compare_4 cases=256
\$finish called at 2560000 (1ps) → 2560ns 종료

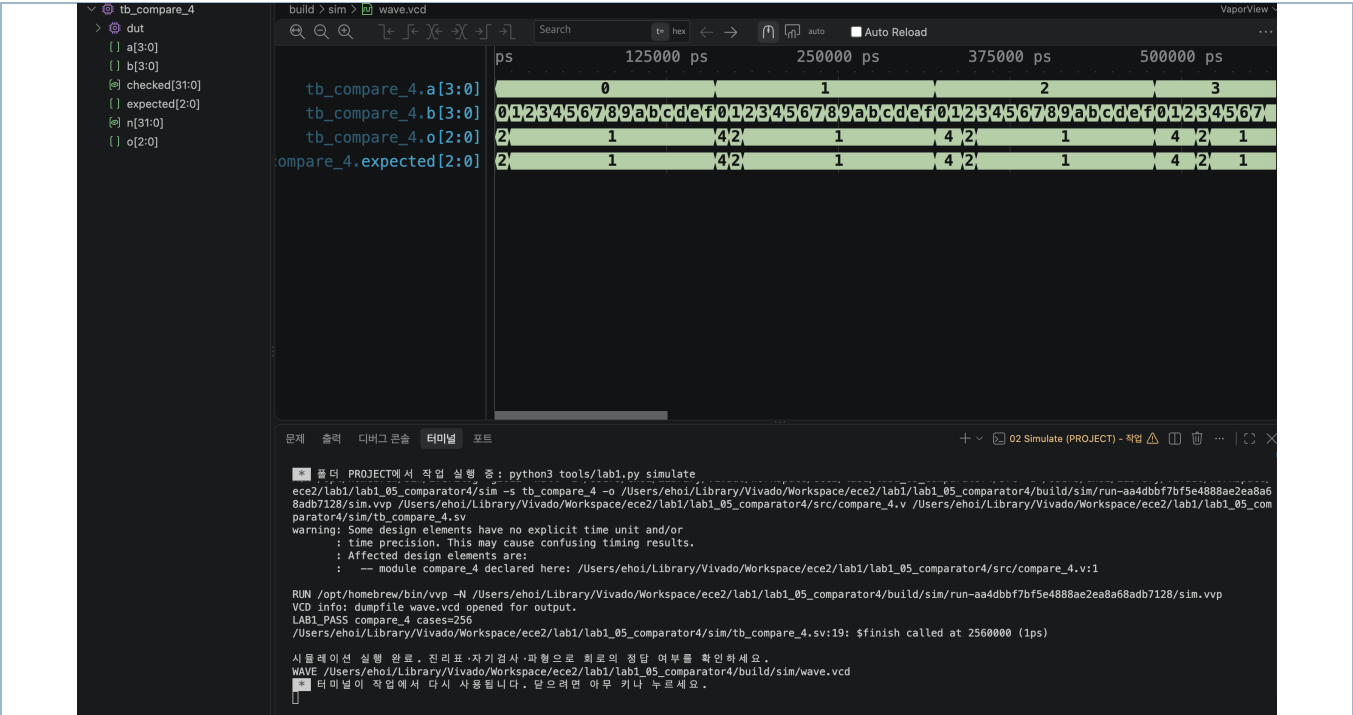


그림 4-5. compare_4 파형 및 실행 로그. 터미널에서 "LAB1_PASS compare_4 cases=256" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a[3:0]	b[3:0]	o[2:0]
5	0000	0000	010
165	0001	0000	100
325	0010	0000	100
485	0011	0000	100
555	0011	0111	001

캡처(사진)에는 o[2:0]뿐 아니라 테스트벤치 내부 expected[2:0] 신호도 함께 표시되어 있고, 두 파형이 전 구간에서 완전히 겹쳐 자기검사 일치를 시각적으로도 확인할 수 있다. 위 시간은 캡처에 실제로 보이는 0~약 560,000ps 구간(전체 2,560,000ps 중 앞부분) 안의 값이다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: compare_4.v의 가운데 비교(a==b)를 a!=b로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: assign o = {a > b, a == b, a < b}; → assign o = {a > b, a != b, a < b};
실패 로그: FAIL compare_4 vector=0 expected=2 actual=0 (Time: 10000) — a=b=0에서 정상 출력은 010(=2, 같음)인데, a!=b로 바뀌며 가운데 비트가 0이 되어 000(실제값 0)으로 실패.

복구 확인: a==b로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS compare_4 cases=256, \$finish called at 2560000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
a[3]	Y1	a[2]	W3
a[1]	U2	a[0]	T1
b[3]	W4	b[2]	W1
b[1]	V4	b[0]	U4
o[2]	L4	o[1]	M4
o[0]	M2		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/compare_4.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_05_comparator4/constraints/compare_4.xdc

보드 확인 예정 입출력: DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=큼(a>b), LED2=같음(a=b), LED3=작음(a<b).

4.6 06 · 4:1 멀티플렉서

설계 내용

포트: 입력 $i[3:0]$, $s[1:0] \rightarrow$ 출력 z (1비트)

동작: $\text{assign } z = i[3-s]$; 선택선 $s=00,01,10,11$ 이 각각 $i[3],i[2],i[1],i[0]$ 을 z 로 내보낸다 (원본 실습의 비트 순서를 그대로 따른다).

저장소 경로: https://github.com/taeyyi/UOS_ece2/tree/master/lab1/lab1_06_mux4to1

src/mux_4x1.v

https://github.com/taeyyi/UOS_ece2/blob/master/lab1/lab1_06_mux4to1/src/mux_4x1.v

```
module mux_4x1(input wire [3:0] i, input wire [1:0] s, output wire z);
    // Preserve the original lab ordering: 00 -> i[3].
    assign z = i[3-s];
endmodule
```

실행 전 예상값

입력	예상 결과
$i=1000, s=00$	$z=1$
$i=1000, s=01$	$z=0$
$i=0001, s=11$	$z=1$

$s=00$ 은 최상위 비트 $i[3]$ 을, $s=11$ 은 최하위 비트 $i[0]$ 을 선택한다.

검사 조건과 테스트벤치

입력 조합: $\{i,s\}=n$ ($n=0..63$)로 i 의 16가지 값과 s 의 4가지 값 조합을 대입

기대값 계산·비교: 기대값 = $((n/4) \gg (3-(n\%4))) \& 1$ 로 s 에 대응하는 i 비트를 계산해 z 와 비교($!=$).

checked=64 확인 후 LAB1_PASS, \$finish.

총 64개 조합 $\times$ 10ns = 640ns에 정상 종료.

sim/tb_mux_4x1.sv

https://github.com/taeyyi/UOS_ece2/blob/master/lab1/lab1_06_mux4to1/sim/tb_mux_4x1.sv

```
`timescale 1ns/1ps
module tb_mux_4x1;
    reg [3:0] i; reg [1:0] s; wire z; mux_4x1 dut(i,s,z);
    integer n,checked=0;
    reg [0:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_mux_4x1);
        for(n=0;n<64;n=n+1) begin
            {i,s}=n;
            expected=((n/4) >> (3-(n%4))) & 1;
            #10;
            if (z != expected) begin
                $fatal(1,"FAIL mux_4x1 vector=%0d expected=%h actual=%h",n,expected,z);
            end
            checked=checked+1;
        end
        if(checked!=64) $fatal(1,"Incomplete test");
        $display("LAB1_PASS mux_4x1 cases=%0d",checked);
        $finish;
    end
end
```

```
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS mux_4x1 cases=64

\$finish called at 640000 (1ps) → 640ns 종료

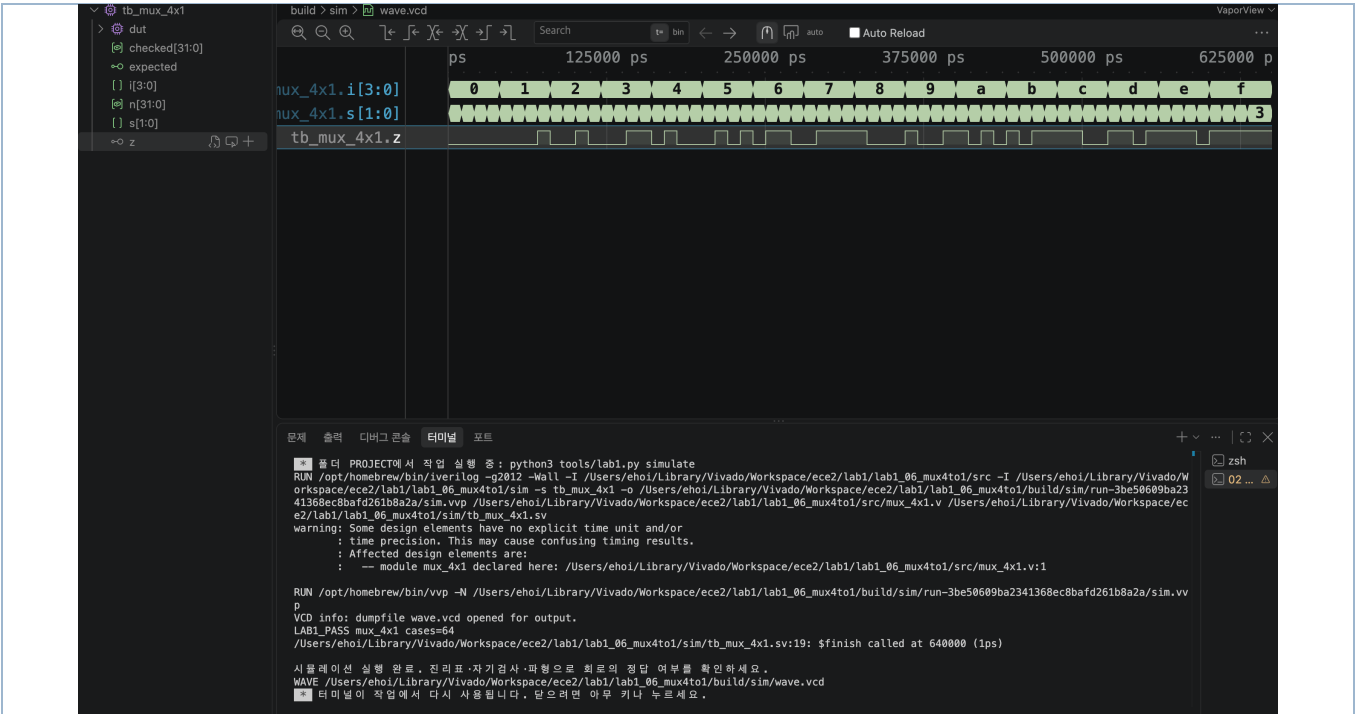


그림 4-6. mux_4x1 파형 및 실행 로그. 터미널에서 "LAB1_PASS mux_4x1 cases=64" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	i[3:0]	s[1:0]	z
5	0000	00	0
15	0000	01	0
35	0000	11	0
325	1000	00	1
635	1111	11	1

캡처(사진)는 전체 640,000ps 중 0~약 630,000ps까지를 보여주며(거의 전 구간), 위 5개 시간 모두 화면에서 직접 확인 가능하다. 635ns는 마지막 케이스(i=1111, s=11)로 화면 가장 오른쪽 끝에 해당한다.

코드 수정 실험

PDF 25쪽의 "코드 수정 실험"을 실제로 수행했다: mux_4x1.v의 출력 인덱스를 3-s에서 s로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: assign z = i[3-s]; → assign z = i[s];

실패 로그: FAIL mux_4x1 vector=4 expected=0 actual=1 (Time: 50000) — i=0001,s=00에서 정상 z=i[3]=0이어야 하는데, 인덱스가 바뀌며 z=i[0]=1이 되어 실패.

복구 확인: i[3-s]로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS mux_4x1 cases=64, \$finish called at 640000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
z	L4	i[3]	K4
i[2]	N8	i[1]	N4
i[0]	N1	s[1]	Y1
s[0]	W3		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/mux_4x1.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_06_mux4to1/constraints/mux_4x1.xdc

보드 확인 예정 입출력: KEY1-4=i[3:0], DIP1-2=s[1:0]. LED1=z.

4.7 07 · 1:8 디멀스

설계 내용

포트: 입력 i(1비트), s[2:0] → 출력 o[7:0]

동작: assign o = i ? (8'b10000000 >> s) : 8'b0; i=0이면 모든 출력이 0이고, i=1이면 s가 가리키는 한 비트만 1이 된다 (s=000→o[7], s=111→o[0]).

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_07_demux1to8

src/demux_1x8.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_07_demux1to8/src/demux_1x8.v

```
module demux_1x8(input wire i, input wire [2:0] s, output wire [7:0] o);
    // Preserve the original lab ordering: 000 -> o[7].
    assign o = i ? (8'b10000000 >> s) : 8'b0;
endmodule
```

실행 전 예상값

입력	예상 결과
i=0, 모든 s	00000000
i=1, s=000	10000000
i=1, s=111	00000001

i가 선택된 한 출력으로만 전달되고 나머지 출력은 항상 0이다.

검사 조건과 테스트벤치

입력 조합: {i,s}=n (n=0..15)로 16가지 조합을 대입

기대값 계산·비교: 기대값 = (n>=8) ? (128 >> (n%8)) : 0. o와 비교(!==). checked=16 확인 후 LAB1_PASS, \$finish.

총 16개 조합 × 10ns = 160ns에 정상 종료.

sim/tb_demux_1x8.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_07_demux1to8/sim/tb_demux_1x8.sv

```
`timescale 1ns/1ps
module tb_demux_1x8;
    reg i; reg [2:0] s; wire [7:0] o; demux_1x8 dut(i,s,o);
    integer n,checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_demux_1x8);
        for(n=0;n<16;n=n+1) begin
            {i,s}=n;
            expected=(n>=8) ? (128 >> (n%8)) : 0;
            #10;
            if (o != expected) begin
                $fatal(1,"FAIL demux_1x8 vector=%0d expected=%h actual=%h",n,expected,o);
            end
            checked=checked+1;
        end
        if(checked!=16) $fatal(1,"Incomplete test");
        $display("LAB1_PASS demux_1x8 cases=%0d",checked);
        $finish;
    end
end
```

```
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS demux_1x8 cases=16

\$finish called at 160000 (1ps) → 160ns 종료

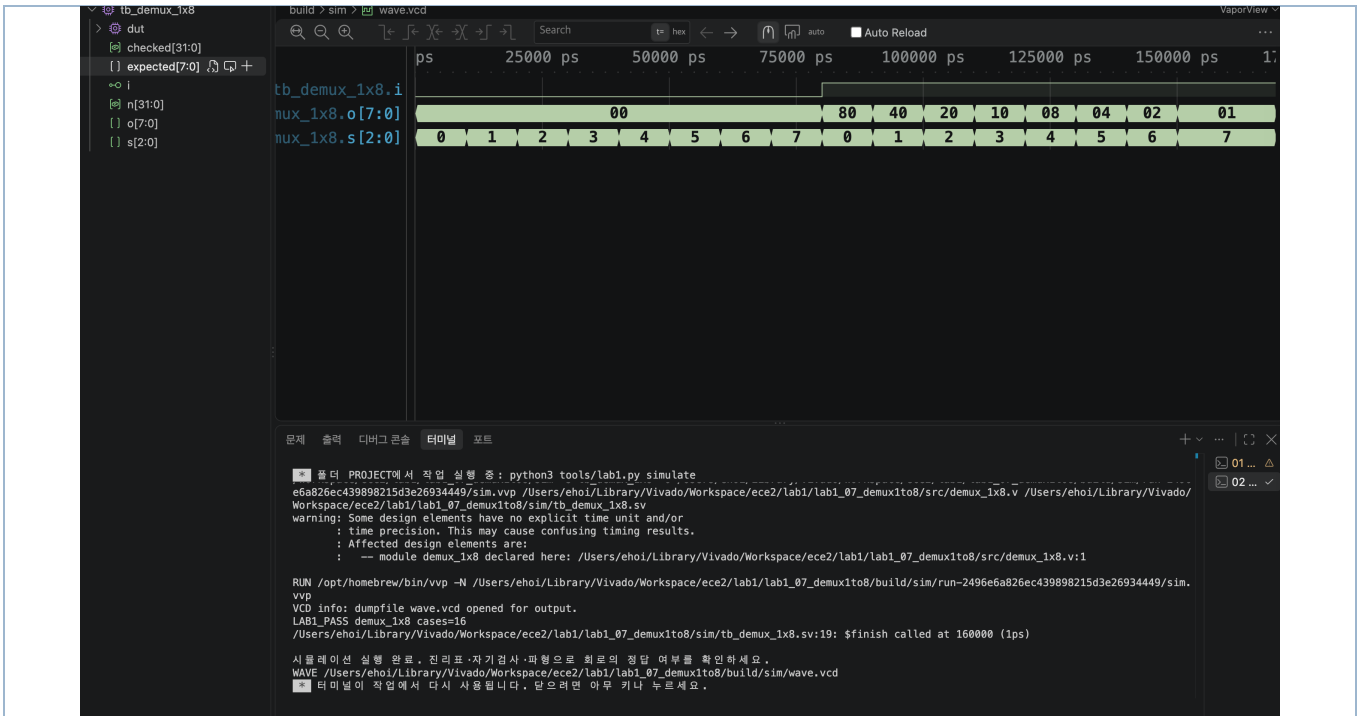


그림 4-7. demux_1x8 파형 및 실행 로그. 터미널에서 "LAB1_PASS demux_1x8 cases=16" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	i	s[2:0]	o[7:0]
5	0	000	00000000
15	0	001	00000000
35	0	011	00000000
85	1	000	10000000
155	1	111	00000001

캡처(사진)는 전체 160,000ps(=\$finish 시각)가 모두 보이므로, 위 5개 시간 전부 화면에서 직접 확인 가능하다. 80~90ns에서 가장 왼쪽 비트, 150~160ns에서 가장 오른쪽 비트가 켜진다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: demux_1x8.v의 오른쪽 시프트를 왼쪽 시프트로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: assign o = i ? (8'b10000000 >> s) : 8'b0; → assign o = i ? (8'b10000000 << s) : 8'b0;

실패 로그: FAIL demux_1x8 vector=9 expected=40 actual=00 (Time: 100000) — i=1,s=001에서 정상 출력은 01000000(=40)인데, 왼쪽 시프트로 바뀌며 8비트를 벗어나 00000000(실제값 00)이 되어 실패.

복구 확인: 오른쪽 시프트로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS demux_1x8 cases=16, \$finish called at 160000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
i	K4	o[7]	L4
o[6]	M4	o[5]	M2
o[4]	N7	o[3]	M7
o[2]	M3	s[2]	Y1
s[1]	W3	s[0]	U2
o[1]	M1	o[0]	N5

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/demux_1x8.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_07_demux1to8/constraints/demux_1x8.xdc

보드 확인 예정 입출력: KEY1=i, DIP1-3=s[2:0]. LED1-8=o[7:0].

4.8 08 · 8:3 인코더

설계 내용

포트: 입력 $i[7:0] \rightarrow$ 출력 $a[2:0]$ (reg)

동작: always@* case(i)로 8개의 one-hot 입력만 번호로 변환한다 ($i[7]=1 \rightarrow 0 \dots i[0]=1 \rightarrow 7$). 0 또는 다중 비트 (multi-hot) 입력은 모두 default로 000을 출력하며, 우선순위 인코더가 아니다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_08_encoder8to3

src/encoder8x3.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_08_encoder8to3/src/encoder8x3.v

```
module encoder8x3(input wire [7:0] i, output reg [2:0] a);
    // One-hot only. Zero and multi-hot inputs return 000 (not a priority encoder).
    always @* begin
        case (i)
            8'h80: a=0; 8'h40: a=1; 8'h20: a=2; 8'h10: a=3;
            8'h08: a=4; 8'h04: a=5; 8'h02: a=6; 8'h01: a=7;
            default: a=0;
        endcase
    end
endmodule
```

실행 전 예상값

입력	예상 결과
10000000	000
00000001	111
00000011	000 (잘못된 입력)

세 번째 예(00000011)처럼 비트가 2개 이상 켜지면 유효하지 않은 입력으로 간주해 000을 출력한다.

검사 조건과 테스트벤치

입력 조합: $i=n$ ($n=0..255$)로 8개 one-hot 값뿐 아니라 256개 입력 전체를 대입

기대값 계산·비교: $k=0..7$ 중 $n==(128 > k)$ 인 경우에만 기대값= k , 그 외에는 0으로 둔다. a와 비교(!==).

checked=256 확인 후 LAB1_PASS, \$finish.

총 256개 조합 $\times$ 10ns = 2560ns에 정상 종료.

sim/tb_encoder8x3.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_08_encoder8to3/sim/tb_encoder8x3.sv

```
`timescale 1ns/1ps
module tb_encoder8x3;
    reg [7:0] i; wire [2:0] a; integer k; encoder8x3 dut(i,a);
    integer n,checked=0;
    reg [2:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_encoder8x3);
        for(n=0;n<256;n=n+1) begin
            i=n;
            expected=0;
            for(k=0;k<8;k=k+1) if(n==(128>>k)) expected=k;
            #10;
            if (a !== expected) begin
                $fatal(1,"FAIL encoder8x3 vector=%0d expected=%h actual=%h",n,expected,a);
            end
        end
    end
endmodule
```

```

end
checked=checked+1;
end
if(checked!=256) $fatal(1,"Incomplete test");
$display("LAB1_PASS encoder8x3 cases=%0d",checked);
$finish;
end
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

시뮬레이션 결과

LAB1_PASS encoder8x3 cases=256

\$finish called at 2560000 (1ps) → 2560ns 종료

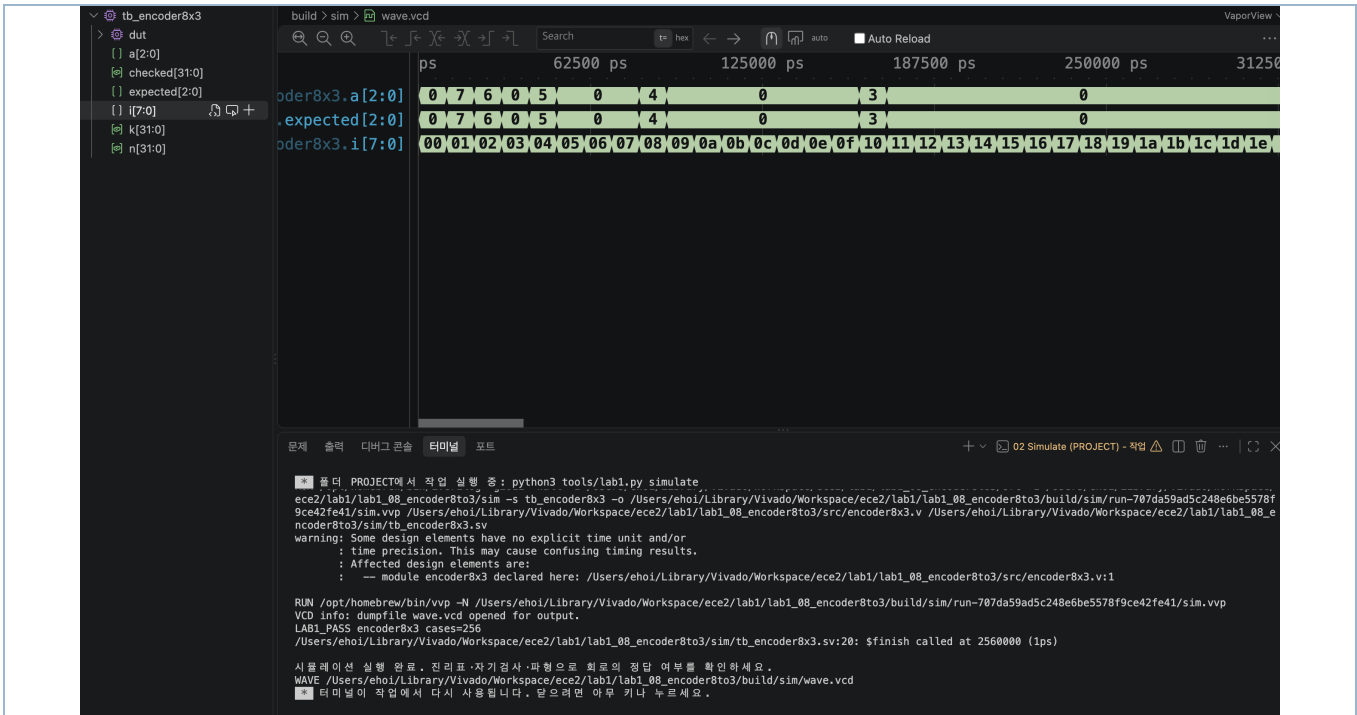


그림 4-8. encoder8x3 파형 및 실행 로그. 터미널에서 "LAB1_PASS encoder8x3 cases=256" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	i[7:0]	a[2:0]
5	00000000	000
15	00000001	111
35	00000011	000
85	00001000	100
165	00010000	011

캡처(사진)에는 `a[2:0]`와 테스트벤치 내부 `expected[2:0]`가 함께 표시되어 있고, 두 파형이 전 구간에서 완전히 겹쳐 자기검사 일치를 시각적으로도 확인할 수 있다. 위 시간은 캡처에 실제로 보이는 0~약 320,000ps 구간(전체 2,560,000ps 중 앞부분) 안의 값이며, 15ns(01)와 35ns(03)를 비교하면 우선순위 인코더가 아님을 확인할 수 있다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: encoder8x3.v에서 입력 00000001(8'h01)에 대응하는 출력 상수를 7에서 6으로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: 8'h01: a=7; → 8'h01: a=6;

실패 로그: FAIL encoder8x3 vector=1 expected=7 actual=6 (Time: 20000) — 입력 00000001의 정상 출력은 111(=7)인데, 상수를 바꾸며 110(실제값 6)이 되어 실패.

복구 확인: 7로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS encoder8x3 cases=256, \$finish called at 2560000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
a[2]	L4	a[1]	M4
a[0]	M2	i[7]	K4
i[6]	N8	i[5]	N4
i[4]	N1	i[3]	P6
i[2]	N6	i[1]	L5
i[0]	J2		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/encoder8x3.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_08_encoder8to3/constraints/encoder8x3.xdc

보드 확인 예정 입출력: KEY1-8=i[7:0]. LED1-3=a[2:0]. 출력 000만으로는 유효 입력 여부를 판단할 수 없다.

4.9 09 · 3:8 디코더

설계 내용

포트: 입력 a, b, c (각 1비트) → 출력 o[7:0]

동작: assign o = 8'b00000001 << {a,b,c}; abc의 이진값과 같은 위치의 비트 하나만 1이 된다 (000→o[0], 111→o[7]).

저장소 경로: https://github.com/taeyi/UOS_ece2/tree/master/lab1/lab1_09_decoder3to8

src/decoder3x8.v

https://github.com/taeyi/UOS_ece2/blob/master/lab1/lab1_09_decoder3to8/src/decoder3x8.v

```
module decoder3x8(input wire a,b,c, output wire [7:0] o);
    assign o = 8'b00000001 << {a,b,c};
endmodule
```

실행 전 예상값

입력	예상 결과
000	00000001
011	00001000
111	10000000

입력이 1 증가할 때마다 켜지는 비트가 한 칸씩 왼쪽으로 이동한다.

검사 조건과 테스트벤치

입력 조합: {a,b,c}=n (n=0..7)로 000~111을 대입

기대값 계산·비교: 기대값 = $1 < n$. o와 비교(!=). checked=8 확인 후 LAB1_PASS, \$finish.

총 8개 조합 × 10ns = 80ns에 정상 종료.

sim/tb_decoder3x8.sv

https://github.com/taeyi/UOS_ece2/blob/master/lab1/lab1_09_decoder3to8/sim/tb_decoder3x8.sv

```
`timescale 1ns/1ps
module tb_decoder3x8;
    reg a,b,c; wire [7:0] o; decoder3x8 dut(a,b,c,o);
    integer n,checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_decoder3x8);
        for(n=0;n<8;n=n+1) begin
            {a,b,c}=n;
            expected=1 << n;
            #10;
            if (o != expected) begin
                $fatal(1,"FAIL decoder3x8 vector=%0d expected=%h actual=%h",n,expected,o);
            end
            checked=checked+1;
        end
        if(checked!=8) $fatal(1,"Incomplete test");
        $display("LAB1_PASS decoder3x8 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule
```

시뮬레이션 결과

LAB1_PASS decoder3x8 cases=8
\$finish called at 80000 (1ps) → 80ns 종료

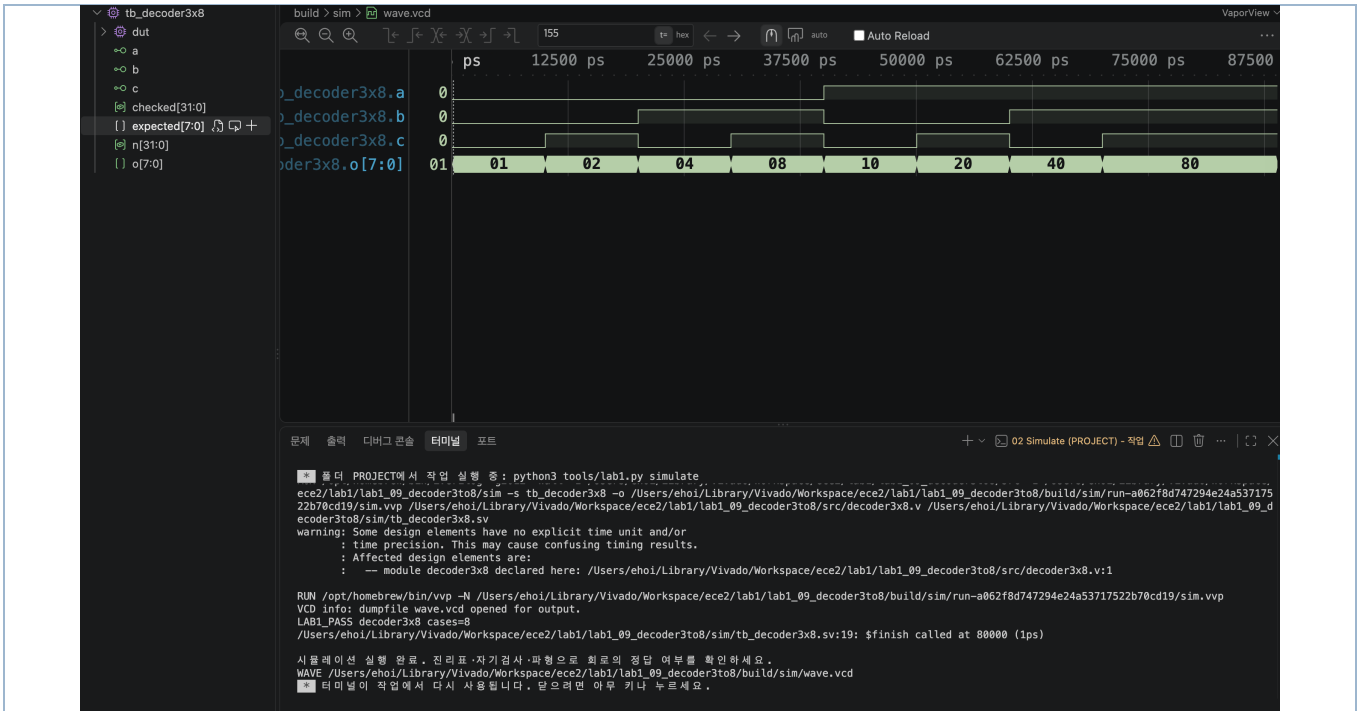


그림 4-9. decoder3x8 파형 및 실행 로그. 터미널에서 "LAB1_PASS decoder3x8 cases=8" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	a	b	c	o[7:0]
5	0	0	0	00000001
15	0	0	1	00000010
35	0	1	1	00001000
45	1	0	0	00010000
75	1	1	1	10000000

캡처(사진)는 전체 87,500ps까지 보이며(전체 80,000ps보다 넓은 범위), 위 5개 시간 전부 화면에서 직접 확인 가능하다.

코드 수정 실험

PDF 26쪽의 "코드 수정 실험"을 실제로 수행했다: decoder3x8.v의 왼쪽 시프트를 오른쪽 시프트로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: assign o = 8'b00000001 << {a,b,c}; → assign o = 8'b00000001 >> {a,b,c};

실패 로그: FAIL decoder3x8 vector=1 expected=02 actual=00 (Time: 20000) — abc=001의 정상 출력은 00000010(=02)인데, 오른쪽 시프트로 바뀌며 비트가 범위를 벗어나 00000000(실제값 00)이 되어 실패.

복구 확인: 왼쪽 시프트로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS decoder3x8 cases=8, \$finish called at 80000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
a	K4	b	N8
c	N4	o[7]	L4
o[6]	M4	o[5]	M2
o[4]	N7	o[3]	M7
o[2]	M3	o[1]	M1
o[0]	N5		

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/decoder3x8.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_09_decoder3to8/constraints/decoder3x8.xdc

보드 확인 예정 입출력: KEY1=a, KEY2=b, KEY3=c. LED1-8=o[7:0]이므로 000에서 LED8, 111에서 LED1이 켜진다.

4.10 10 · 7 세그먼트 디코더

설계 내용

포트: 입력 bcd[3:0] → 출력 seg_data[7:0] ({a,b,c,d,e,f,g,dp} 순서, 1=점등, dp는 항상 0)

동작: always@* case(bcd)로 0~F 16개 입력 각각에 대응하는 세그먼트 점등 패턴을 8비트 상수로 출력한다.

저장소 경로: https://github.com/tae2yi/UOS_ece2/tree/master/lab1/lab1_10_seven_segment

src/seg_decoder.v

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_10_seven_segment/src/seg_decoder.v

```
module seg_decoder(input wire [3:0] bcd, output reg [7:0] seg_data);
    // [7:0] = {a,b,c,d,e,f,g,dp}; active high; hexadecimal 0..F; dp off.
    always @* begin
        case (bcd)
            0:seg_data=8'hfc; 1:seg_data=8'h60; 2:seg_data=8'hda; 3:seg_data=8'hf2;
            4:seg_data=8'h66; 5:seg_data=8'hb6; 6:seg_data=8'hbe; 7:seg_data=8'he0;
            8:seg_data=8'hfe; 9:seg_data=8'hf6; 10:seg_data=8'hee; 11:seg_data=8'h3e;
            12:seg_data=8'h9c; 13:seg_data=8'h7a; 14:seg_data=8'h9e; 15:seg_data=8'h8e;
            default:seg_data=0;
        endcase
    end
endmodule
```

실행 전 예상값

입력	예상 결과
0	FC: a,b,c,d,e,f
1	60: b,c
8	FE: a-g
F	8E: a,e,f,g

8은 일곱 획이 모두 켜지고(FE), dp 비트는 항상 0으로 꺼져 있다.

검사 조건과 테스트벤치

입력 조합: bcd=n (n=0..15)로 0~F를 대입

기대값 계산-비교: 교안에서 정의한 0~F의 기대 점등 패턴 16개를 TB의 digits[0:15] 배열에 별도로 정의하고, 이를 기대값으로 실제 seg_data와 비교(!==)한다. checked=16 확인 후 LAB1_PASS, \$finish.

총 16개 조합 × 10ns = 160ns에 정상 종료.

sim/tb_seg_decoder.sv

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_10_seven_segment/sim/tb_seg_decoder.sv

```
`timescale 1ns/1ps
module tb_seg_decoder;
    reg [3:0] bcd; wire [7:0] seg_data; reg [7:0] digits[0:15]; seg_decoder dut(bcd,seg_data);
    integer n,checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_seg_decoder);
        digits[0]=8'hfc;
        digits[1]=8'h60;
        digits[2]=8'hda;
        digits[3]=8'hf2;
        digits[4]=8'h66;
    end
```

```

digits[5]=8'hb6;
digits[6]=8'hbe;
digits[7]=8'he0;
digits[8]=8'hfe;
digits[9]=8'hf6;
digits[10]=8'hee;
digits[11]=8'h3e;
digits[12]=8'h9c;
digits[13]=8'h7a;
digits[14]=8'h9e;
digits[15]=8'h8e;

for(n=0;n<16;n=n+1) begin
    bcd=n;
    expected=digits[n];
    #10;
    if (seg_data != expected) begin
        $fatal(1,"FAIL seg_decoder vector=%0d expected=%h actual=%h",n,expected,seg_data);
    end
    checked=checked+1;
end
if(checked!=16) $fatal(1,"Incomplete test");
$display("LAB1_PASS seg_decoder cases=%0d",checked);
$finish;
end
initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

시뮬레이션 결과

LAB1_PASS seg_decoder cases=16

\$finish called at 160000 (1ps) → 160ns 종료

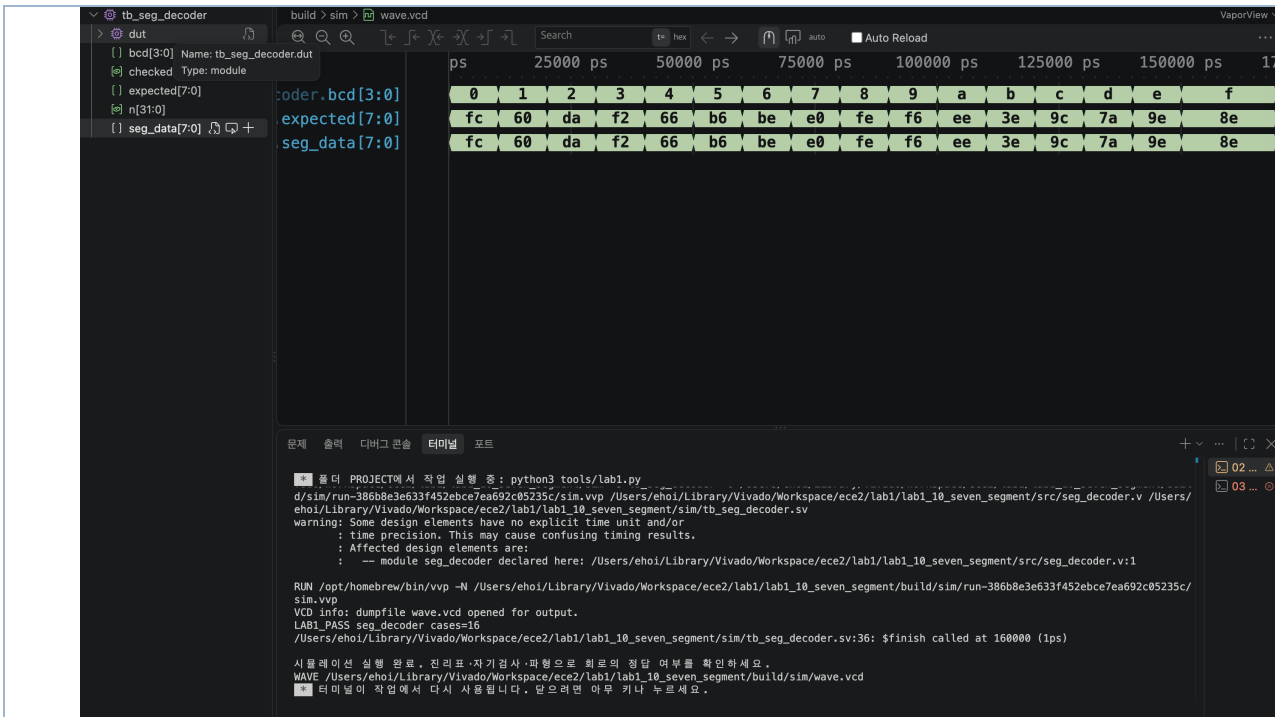


그림 4-10. seg_decoder 파형 및 실행 로그. 터미널에서 "LAB1_PASS seg_decoder cases=16" 문구를 직접 확인했다.

시간 구간별 값:

시간(ns)	bcd[3:0]	seg_data[7:0]
5	0000	11111100
15	0001	01100000
35	0011	11110010

시간(ns)	bcd[3:0]	seg_data[7:0]
85	1000	11111110
155	1111	10001110

캡처(사진)에는 seg_data[7:0]와 테스트벤치 내부 expected[7:0]가 함께 표시되어 있고, 두 파형이 전 구간에서 완전히 겹쳐 자기검사 일치 여부를 시각적으로도 확인할 수 있다. 전체 160,000ps(=\$finish 시각)가 모두 보이므로 위 5개 시간 전부 화면에서 직접 확인 가능하다.

코드 수정 실험

PDF 29쪽의 "코드 수정 실험"을 실제로 수행했다: seg_decoder.v에서 숫자 0의 패턴을 1의 패턴으로 바꾼 뒤 저장하고 02 Simulate를 재실행했다.

변경: 0:seg_data=8'hfc; → 0:seg_data=8'h60;

실패 로그: FAIL seg_decoder vector=0 expected=fc actual=60 (Time: 10000) — bcd=0의 정상 패턴은 FC인데, 1의 패턴(60)으로 바뀌어 실패.

복구 확인: FC로 복구하고 Save All → 02 Simulate 재실행 → LAB1_PASS seg_decoder cases=16, \$finish called at 160000 (1ps)로 정상 종료 확인.

핀 배정 (XDC)

포트	PACKAGE_PIN	포트	PACKAGE_PIN
bcd[3]	Y1	bcd[2]	W3
bcd[1]	U2	bcd[0]	T1
seg_data[7]	P1	seg_data[6]	P3
seg_data[5]	P7	seg_data[4]	N3
seg_data[3]	T5	seg_data[2]	R2
seg_data[1]	R4	seg_data[0]	R6

모든 포트의 IOSTANDARD는 LVCMOS33(3.3V CMOS)으로 지정했다.

constraints/seg_decoder.xdc

https://github.com/tae2yi/UOS_ece2/blob/master/lab1/lab1_10_seven_segment/constraints/seg_decoder.xdc

보드 확인 예정 입출력: DIP1-4=bcd[3:0]. 단일 7세그먼트 a-dp. 이름은 bcd지만 입력 10-15도 A,b,C,d,E,F로 정의한다.

5. 결론

5.1 전체 검증 결과

10개 회로 모두 전체 입력 조합에 대해 자기검사를 통과했다. 각 회로의 검사 규모와 결과는 다음과 같다.

Lab	설계 top	검사 조합 수	종료 시간	결과
01	logic_gate	4개	40ns	PASS
02	full_adder	8개	80ns	PASS
03	adder_4bit	256개	2560ns	PASS
04	sub_4bit	256개	2560ns	PASS
05	compare_4	256개	2560ns	PASS
06	mux_4x1	64개	640ns	PASS
07	demux_1x8	16개	160ns	PASS
08	encoder8x3	256개	2560ns	PASS
09	decoder3x8	8개	80ns	PASS
10	seg_decoder	16개	160ns	PASS

검사 조합 수는 회로의 입력 비트 수에 따라 결정된다. 입력이 8비트인 4비트 가산기·감산기·비교기·인코더는 $2^8=256$ 개 조합 전수 검사를 수행했고, 나머지 회로도 가능한 입력 조합을 모두 인가했다. 즉 표본 검사가 아니라 전수 검사이므로 미검증 입력 조합은 존재하지 않는다.

5.2 코드 수정 실험으로 확인한 검사의 유효성

10개 실습 모두에서 설계 RTL을 한 줄 수정해 오류를 주입했을 때 테스트벤치가 예외 없이 실패를 검출했다. 예를 들어 Lab 01에서 XOR를 OR로 바꾸자 $a=b=1$ 조합에서 기대값 110과 실제값 111의 불일치를 검출해 중단했고, Lab 06에서 인덱스를 3-s에서 s로 바꾸자 $i=0001, s=00$ 조합에서 오류가 드러났다. 이는 각 테스트벤치가 통과 문구를 출력하는 것 자체가 의미 있는 검증 결과임을 뒷받침한다.

5.3 검증된 범위와 검증되지 않은 범위

본 단계에서 확인한 것과 아직 확인하지 못한 것을 구분하면 다음과 같다.

구분	내용
검증됨	설계 RTL의 논리적 기능. 모든 입력 조합에서 출력이 기대값과 일치함을 확인했다.

구분	내용
검증됨	파형상의 시간 구간별 값이 예상 진리표와 일치함을 시각적으로 확인했다.
미검증	핀 배정의 정확성. XDC는 기능 시뮬레이션에서 읽지 않으므로 Vivado 구현과 실제 보드에서 확인해야 한다.
미검증	합성 후 타이밍과 전파 지연. 본 시뮬레이션은 게이트 지연을 고려하지 않는 기능 시뮬레이션이다.
미검증	실제 장치 동작. 보드 기록(Program Device)과 스위치·LED 조작 결과는 2부·3부에서 확인한다.

5.4 고찰

이번 실습에서 얻은 주요 시사점은 다음 세 가지다.

- 도구의 실행 성공(SIMULATED)과 기능의 정확성(PASS)은 별개다. 파형이 생성되었다는 사실만으로는 회로가 옳다고 판단할 수 없으며, 기대값을 독립적으로 계산해 비교하는 자기검사가 있어야 판정이 가능하다.
- 인코더처럼 유효 입력이 제한된 회로는 출력만으로 입력의 유효성을 판단할 수 없는 구조적 한계를 가질 수 있다. Lab 08 에서 무입력·정상 입력·다중 입력이 모두 000 으로 나타나는 현상이 그 예이며, 실제 설계에서는 유효 신호를 별도로 두어 이를 해결한다.
- 동일한 개념의 회로라도 구현 세부에 따라 동작이 달라진다. Lab 07 의 디멀스와 Lab 09 의 디코더는 모두 3 비트 주소로 8 개 중 하나를 선택하지만 시프트 방향이 반대여서 같은 주소에 대해 서로 다른 출력 비트가 켜진다. 개념적 유사성만 믿고 구현을 확인하지 않으면 오류로 이어질 수 있다.

다음 단계에서는 동일한 소스를 Vivado 프로젝트로 가져와 합성·구현·비트스트림 생성을 수행하고, 본 보고서에서 작성한 XDC의 핀 배정이 실제 보드의 스위치·LED와 일치하는지 확인한다.

6. 참고문헌

- [1] 서울시립대학교 전기전자컴퓨터공학부, "04.LAB1_01_LOGIC_GATES_VIVADO", 전기전자설계실험 2 강의자료, 2026.
- [2] 서울시립대학교 전기전자컴퓨터공학부, "04.LAB1_02_FULL_ADDER_VIVADO", 전기전자설계실험 2 강의자료, 2026.
- [3] 서울시립대학교 전기전자컴퓨터공학부, "04.LAB1_03_ADDER4_VIVADO" ~ "04.LAB1_10_SEVEN_SEGMENT_VIVADO", 전기전자설계실험 2 강의자료, 2026.
- [4] Glaysia, "fpga-lab-template" (v2.0.0), GitHub repository. <https://github.com/Glaysia/fpga-lab-template>
- [5] S. Williams, "Icarus Verilog" (version 13.0), <https://steveicarus.github.io/iverilog/>
- [6] L. Ramseyer, "VaporView — VCD waveform viewer for VS Code", Visual Studio Marketplace.
- [7] IEEE, "IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language", IEEE Std 1800-2017.
- [8] AMD Xilinx, "Vivado Design Suite User Guide: Using Constraints" (UG903).

본 실습의 소스 코드 및 증빙 자료 저장소:

https://github.com/tae2yi/UOS_ece2/tree/master/lab1